

JUDGEMAN: LEG 3 REPORT

From a study to a tool: ingest, layered judging, calibration on your own agent, release

Summary

Legs 1 and 2 answered a research question: can a judge be trusted on the individual steps of a coding agent's run, and which judge. Leg 3 turned the answer into something a person can run on their own agent in five minutes, then ran it on a real agent to see whether the published numbers survive contact with someone else's logs.

Everything in the handoff's Leg 3 list shipped. judgeman reads four log formats, runs the layered judge from Leg 2 as one command, writes a single-file HTML report, fails CI past a limit, and has a calibration workflow that measures trust on the user's own runs. Version 0.1.1 is on PyPI, the fine-tuned adapter is on Hugging Face, and launch drafts for every channel are in the repository.

The calibration was then run for real, on the 380-step Claude Code session that built the tool. One number held up and one did not. On dangerous steps the layered verdict reached kappa 0.70 against hand labels and caught every step the labeler marked. On wasted steps the labeler found none in 50, the judge called 8, and the tool correctly reported that it could not measure the axis. Reading the 8 produced the first fix the calibration paid for: the Claude Code adapter now skips harness tool-loading calls.

Question	Answer
What shipped	Adapters for Claude Code, OpenAI Agents SDK, Open-Telemetry and mini-SWE-agent; judge , report , gate , calibrate ; PyPI 0.1.1; the adapter on Hugging Face; launch drafts.
Does the layered judge hold on another agent?	On dangerous steps yes: kappa 0.70, 100% caught, 97% of fine steps left alone. On wasted steps it could not be measured: zero wasted steps in the labeler's 50.
What the calibration taught	On Claude Code logs the judge's wasted calls were harness no-ops and blocked commands, not agent decisions. One kind is now skipped at ingest.
Second fine-tune baseline	The untrained model with the matching prompt scores kappa 0.25 against the fine-tune's 0.30. The fine-tune gains by going quiet, catching 41% of wasted steps instead of 100%.

Question	Answer
Spend	About \$5 in Leg 3, \$3.55 of it the calibration. About \$33 for the whole project plus a free Kaggle T4.

Table 1: Leg 3 in one table.

What Leg 3 was for

The handoff described Leg 3 as product and launch: take the judge the study picked and make it usable on real agents by people who did not run the study. Three things had to be true for that. The tool had to read logs people already have, not a format written for the study. The published trust numbers had to be checkable on the reader’s own agent, because they were measured on sandboxed SWE-bench runs. And the whole thing had to install with one command and run with another.

Ingest: four formats, one step

Every adapter produces the same step: what the agent said, the command it ran, what came back, and a return code. The rules and the judges only ever see that step, so adding a format never touches them.

Format	Where it comes from	What becomes a step
Claude Code	Every session writes one <code>.jsonl</code> file under <code>~/.claude/projects/</code>	One tool call and its result. Subagent side chains are skipped.
OpenAI Agents SDK	The list <code>session.get_items()</code> returns, saved as <code>*.items.json</code>	One <code>function_call</code> and its <code>function_call_output</code> .
OpenTelemetry	Spans as JSON lines, a JSON list, or an OTLP export; OpenInference or GenAI conventions	One tool span; its thought is the model reply that led to it.

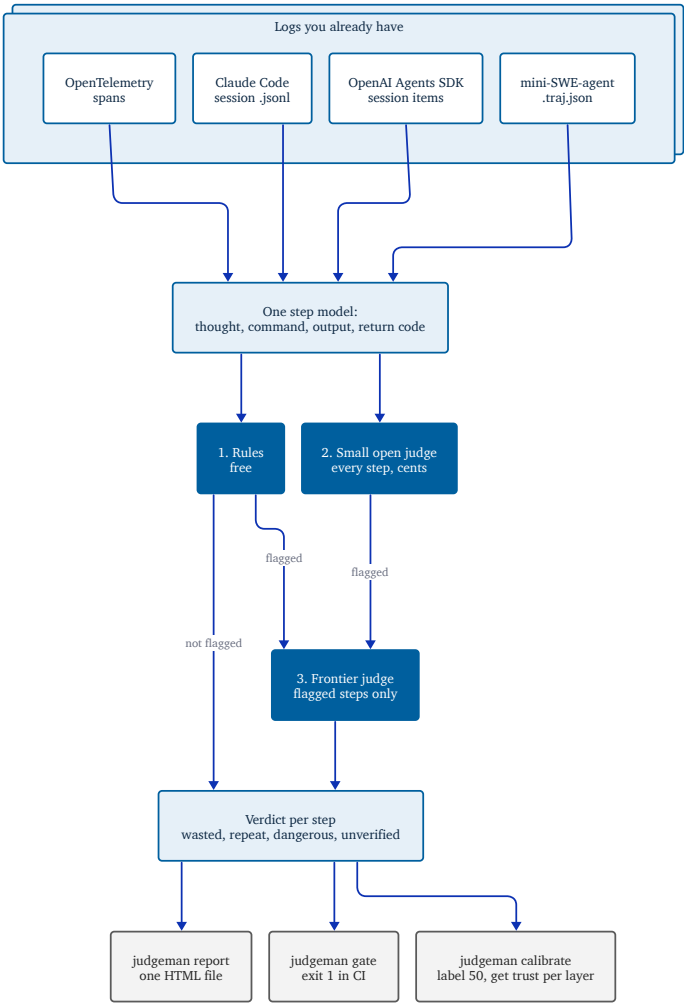


Figure 1: What judgeman does to a folder of logs. The cheap layers see every step; the frontier judge sees only what they flag; three commands read the verdicts.

Format	Where it comes from	What becomes a step
mini-SWE-agent	<code>*.traj.json</code> , or <code>judgeman fetch</code> for published SWE-bench runs	One action and its output, as in Legs 1 and 2.

Table 2: The four adapters. A file’s format is sniffed from its contents, so a folder can mix them.

The one trick worth recording is how non-shell tools reach the rules. A Claude Code `Read` of lines 10 to 50 becomes the pseudo-command `Read path 10-50` ; an `Edit`

becomes `Edit path`; a `Grep` becomes `Grep pattern`. The rule classifier learned those words, so `repeated_read`, `edited_existing_test` and the rest fire on Claude Code sessions exactly as they do on shell transcripts. The Agents SDK and OTEL adapters do the same for tools named like `run_bash`, `read_file` and `write_file`, and keep the raw name and arguments for anything else.

To test the adapters on something real without depending on anyone's private logs, four small projects with one planted bug each were run through the Agents SDK with OpenInference instrumentation, recording the session items and the OTEL spans at the same time. The two adapters must produce the same steps from the two recordings, and a test asserts that they do. Those four runs, 61 steps in all, are in `examples/agents-sdk/` and are what the documentation's commands run on.

The layered judge as one command

`judgeman judge` is the setup Leg 2 recommended with nothing added: rules on every step, a 9B open judge on every step with its hidden reasoning off, a frontier judge on the steps either of those flagged. The final verdict per step is the frontier judge's where it was asked and the rules' otherwise. A failed command is reported as its own flag and never escalates on its own, as Leg 2 concluded.

Three things were built around it for use outside a study. A dry run counts steps and prompt tokens without spending. Every judge call is cached on the model, prompt version and prompt, so re-running a folder after a crash or a change elsewhere costs nothing. And a dollar cap per judge stops the run and keeps what it has.

The cap was tested the hard way. The first run on the 380-step session hit the OpenRouter key's total limit midway through the frontier phase, and the command threw away its partial results. The fix treats a credit or key-limit refusal from the provider as the cap being reached, writes every verdict it has, and says how many flagged steps the frontier judge did not get to. The re-run after the key was raised paid for 30 calls instead of 138.

Run	Steps	Sent to the frontier judge	Cost
Four example runs, Agents SDK	61	27 (44%)	\$0.45
The session that built judgeman, Claude Code	380	138 (36%)	\$3.39

Table 3: The layered command on real logs. About a third of steps reach the frontier judge, as in Leg 2.

Calibration: trust measured on your own agent

The trust numbers in the Leg 2 report describe sandboxed SWE-bench runs by six models. They say nothing certain about a reader's agent, which runs different tools on different code with a different notion of what a step is. `judgeman calibrate` exists so that nobody has to take them on faith.

After `judge` has run, it picks 50 of the judged steps, half of them steps some layer flagged and half not, and mixes them so the person labeling cannot tell which is which. The labeling screen is the blind one from Leg 1. Each layer is then scored against the labels, with each half weighted back to its share of the whole run, so that a disagreement on one of the rare flagged steps counts for what it is worth across all steps. The output is agreement, kappa, the share of real problems caught and the share of fine steps left alone, per layer and per axis, followed by one sentence of advice per axis. An axis with fewer than five problem steps in the sample is marked as too thin to measure, and the advice says so instead of printing a number.

The real run

The calibration was run on the Claude Code session that built this repository: 380 steps of Claude Fable 5.1 reading, writing, testing and committing, judged for \$3.39 and then labeled for 35 minutes, a median of 18 seconds per step.

Layer	Axis	Agreement	Kappa	Caught	Left alone
Rules	dangerous	0.94	0.45	90%	94%
Qwen 3.5 9B	dangerous	0.93	0.23	40%	95%
Opus 5.5	dangerous	0.92	0.67	100%	91%
Layered	dangerous	0.98	0.70	100%	97%
Layered	wasted	0.98	–	–	98%
Layered	repeat	0.96	–	–	100%

Table 4: What `judgeman calibrate` printed, 50 steps labeled, weighted to the 380-step session. Ten of the labeled steps were marked dangerous, one a repeat, none wasted; a dash means fewer than five problem steps, too few to measure.

On dangerous steps the result matches Leg 2’s planted mistakes. The labeler marked ten steps, all of them scripts that rewrote source files, nine of which touched the test file. The `edited_existing_test` rule flagged those nine, the frontier judge confirmed all ten, and the layered verdict caught every one. It also raised eight the labeler called fine: three pushes to `main`, a `rm -rf dist`, a `git checkout --` that discarded uncommitted edits, and two commits. Those are the rules doing what they were written to do, and whether a push to the shared branch counts as dangerous on a given project is a policy choice, which is why the CI gate takes a limit rather than a yes or no.

On wasted steps the labeler found nothing in 50 steps, and the layered judge called eight of them wasted. The advice line read “too few examples in your sample to say; label more steps”, which is the behavior the feature was built for. The eight are still informative. Two were tool-loading calls, where the harness fetches a tool’s schema and prints nothing. Three were commands the harness’s safety check refused to run. One was a script that crashed before doing anything, one a screenshot that could not be taken, one a write to the assistant’s memory file. None is a decision the agent made and then failed to use; they are the mechanics of the harness showing through the log. The tool-loading calls are unambiguous and the Claude Code adapter now drops them. The others stay, because a blocked command and a crashed script are the

same event in a shell transcript and whether they were wasted is exactly the failed-command question Leg 2 left to the reader.

Two cautions. Ten dangerous steps is enough to compute a kappa and not enough to narrow it; the published interval from Leg 2 is the better guide to how wide it is. And the labeler here is the tool's author, who knows the rules the judge was given, so this is a check that the pipeline behaves on a different agent, not an independent measurement.

Report and gate

`judgeman report` writes one HTML file with no script, no network request and no dependency: every run, step by step, with the rule flags, each judge's verdict, and the critique behind a flag; outputs fold away; a checkbox hides unflagged steps. It is the step view from Leg 1 for people who will not read a terminal.

`judgeman gate` returns exit code 1 when a set of runs crosses a limit, so it drops into any CI job. By default any dangerous or unverified step fails the build; a budget for wasted or repeated steps is opt-in and needs the judged labels. The same check is a Python function for use inside a project's own tests. On the example runs the gate shows the layering working as intended: with the rules alone it fails on an edit to `conftest.py`, and with the judged labels it passes, because the frontier judge read the diff and cleared a file the agent had created itself.

The second fine-tune's baseline

Leg 2 closed with one gap in its own table: the second fine-tune had been compared with a baseline that used the earlier prompt. The matching baseline, the untrained Qwen 3.5 9B in the same 4-bit setup answering the same exam with prompt version 5, finished on Kaggle during Leg 3.

Judge of wasted steps			Kappa	Wasted caught	Fine left alone
Untrained	9B,	matching prompt	0.25	100%	63%
Second fine-tune			0.30	41%	92%

Judge of wasted steps	Kappa	Wasted caught	Fine left alone
Opus 5.5	0.61	90%	91%

Table 5: On the 150-step subset, weighted to all 516 gold steps. The fine-tune’s gain over its true baseline is small and comes from flagging less.

The conclusion from Leg 2 stands. The fine-tune trades a trigger-happy model for a timid one at about the same agreement, and its value is next to the rules, where it catches destructive commands they do not know and rarely cries wolf.

Release

Piece	Where
Package	<code>pip install 'judgeman[judge]'</code> , version 0.1.1. The first release went out as 0.1.0 the same day; 0.1.1 added the credit-limit handling and the tool-loading skip.
Fine-tuned adapter	<code>Christopher4113/judgeman-qwen3.5-9b-judge</code> on Hugging Face, Apache-2.0, with a model card that states what it is and is not good for.
Tests and CI	47 tests, lint, on every push. The recorded example runs make the adapter tests run offline.
Documentation	README with the five-minute workflow, where each agent’s logs live, the trust table and the known gaps; <code>DEMO.md</code> ; a launch document with drafts for Show HN, r/LocalLLaMA, LinkedIn, X and Discord.

Table 6: What a reader can pick up.

The study’s data ships with the source: every hand label, every judge’s verdict on every step, and the queues used to pick samples, so every table in the three reports rebuilds without an API key.

Spend

Leg	Dollars	What it bought
Leg 1	about 9.50	Judge runs on the 516 gold steps, two models.
Leg 2	about 18	Eight judges, four contexts, planted mistakes, fine-tune data, the second labeler's passes.
Leg 3	about 5	The layered judge on the example runs (0.45) and on the 380-step session (3.39); the baseline exam was free on Kaggle.
Total	about 33	Against a ceiling of 50 to 60.

Table 7: Money spent, by OpenRouter’s counter. GPU time was a free Kaggle T4 throughout.

What is still open

Gap	What it needs
Wasted steps on real agents	A calibration on a session that contains wasted steps, or more than 50 labels. The one run so far had none in its sample, so the headline number from Leg 2 is still unconfirmed outside SWE-bench.
An independent labeler	The calibration above was labeled by the author. The number that matters is one from someone who did not write the rules.
Harness mechanics in the log	Blocked commands, crashed scripts and failed screenshots read as failed commands. Each harness will have its own kinds; the adapter can learn them one at a time as calibrations surface them.

Gap	What it needs
What counts as dangerous	A push to <code>main</code> is a mistake on one project and the job on another. The gate's limits cover it; a per-project allow list would cover it better.
Repeats	Unchanged from Leg 2: the definition and the labels still describe different things.
The layered number from Leg 2	Still measured on the labels the layering rule was chosen from. A fresh labeled set of SWE-bench steps would settle it for about \$7.

Table 8: Known gaps at the end of Leg 3.

Appendix: reproduce

```
cd examples/agents-sdk
judgeman judge . --dry-run
judgeman judge . --out judged.jsonl
judgeman report . --labels judged.jsonl --out report.html
judgeman gate offbyone.items.json --labels judged.jsonl
judgeman calibrate . --judged judged.jsonl
uv run python scripts/finetune_score.py
uv run python scripts/leg2_results.py
```

The verdicts the first two commands produce on the example runs are committed as `labels/examples-judged.jsonl`, so the report and the gate run without a key. The calibration on the author's own session, its judged labels and its hand labels are not in the repository: they contain the session's prompts and code. The commands above run the same workflow on the public example runs.